Course Video Topic Search (RAG System)

Updated Technical Interview Master Guide | End-to-End Deep Dive in Simple English

Candidate: Anwar Ahmad	**Target Roles:** Data Scientist | ML Engineer | GenAI Developer
Live Deployment: https://topic-search-system.onrender.com	**Corpus:** 52 Video Modules | 2,800 Semantic Chunks
GitHub: Anwar-Ahmad31/course-video-topic-search--RAG-	**Core Stack:** FastAPI, BM25 Okapi, Groq Cloud, Whisper ASR

1. How to Introduce Your Project in an Interview

Use these scripted answers depending on whether the interviewer asks for a quick 30-second summary or a detailed 2-minute walkthrough.

30-Second Elevator Pitch:
*"I built **Course Video Topic Search**, an end-to-end RAG system that indexes over 50+ hours of Python video lectures across 52 modules. It allows learners to ask any programming question in plain English and returns a structured, code-grounded explanation along with the **exact video module and timestamp citations** (e.g., Module 24: 04:12 - 05:30). It saves students from scrubbing through hours of video playlists."*

The Real-Life Analogy (To explain to anyone instantly):
*"Think of a 500-page textbook. If you want to find 'Tuples', you don't read every page — you check the **Index page** at the back, which points you directly to Page 120. Our system acts as that **Index Page for video courses**: it listens to all 52 videos, notes who said what and at what second, and when you ask a question, it hands you the exact 1-minute clip and a direct explanation in under 2 seconds."*

2. Technical Architecture — The 4-Pillar Pipeline

Walk the interviewer through the four clear stages from raw video to the browser UI:

Stage 1: Speech-to-Text Ingestion	Extracted audio tracks (.mp3) from all 52 video lectures. Transcribed spoken bilingual audio into clean text using **OpenAI Whisper**, generating sentence-level start and end timestamps.
Stage 2: Chunking & Noise Pruning	Merged transcript segments into **2,800 semantic chunks** (30–60s each). Built automated filters to prune Whisper silence loops (e.g., repeated hallucinated words) and phonetic drift.
Stage 3: BM25 Hybrid Retrieval Engine	Built a low-memory **BM25 Okapi** index with **Title Weighting (4x boost)**. Resolves complex questions via query decomposition so multi-topic inquiries never starve.
Stage 4: Cloud LLM Synthesis & Citations	Grounded prompts are sent to **Groq Cloud API** (*openai/gpt-oss-120b* with fallback to *qwen/qwen3.8-27b*). Emits Markdown answers with runnable code and clickable video timestamps.

3. Top 4 Engineering Challenges Solved (Interview Winning Points)

Interviewers love when you discuss difficult bugs you diagnosed and fixed yourself. Here are your top four talking points:

Challenge 1: The Compound Question Starvation Bug ('What is set and tuple?')

The Problem: When asking compound questions like *'What is set and tuple?'*, the engine only answered 'Set' and said 'Tuple not found'. This happened because a global *top_k=4* was consumed entirely by the first topic.
The Solution: Implemented a **Query-Stem Rebuilder** that splits compound sentences on connectors ('and', 'plus') and reconstructs focused sub-queries: ['What is set', 'What is tuple']. Allocated an **independent top_k chunk budget per topic**, ensuring both topics get fair evidence.

Challenge 2: The 'Python as a Stop-Word' Trap ('What is Python?')

The Problem: Asking *'What is Python?'* returned *'No technical documentation found'* because 'python' had been placed in a static stop-word set. Removing 'what', 'is', and 'python' left an empty query vector.
The Solution: Replaced rigid stop-word lists with statistical **BM25 Okapi ranking**. Technical keywords ('python', 'loops', 'classes') are never deleted. Cleaned repetitive title boilerplate (| Python Tutorial) across all 52 videos so Module 1's true subject ('Introduction to Programming & Python') stood out with high relevance.

Challenge 3: Low-Memory Cloud Deployment (512MB RAM on Render Free Tier)

The Problem: In-memory dense vector models (like BAAI/bge-m3 ONNX) require ~2.2 GB of RAM, causing Out-Of-Memory (OOM) container crashes on 512MB cloud instances.
The Solution: Developed an ultra-lightweight, pure Python **BM25 Okapi search engine** directly inside the backend. It requires only **~3 MB of RAM**, boots instantly in 0.1s, and retrieves top chunks in **under 15 milliseconds**.

Challenge 4: High-Availability Cloud Inference & Model 404s

The Problem: Certain open-source Groq models occasionally get decommissioned or return 404 depending on cluster capacity.
The Solution: Engineered a **Multi-Model Cascade Fallback**: openai/gpt-oss-120b → openai/gpt-oss-20b → qwen/qwen3.8-27b with automatic retry and local Ollama daemon fallback for offline dev.

4. Key Technical Interview Questions & Answers

Direct, clear answers prepared for technical screenings and hiring manager rounds:

Q1. Why did you choose RAG instead of Fine-Tuning an LLM for this project?

"Fine-tuning changes how a model behaves or speaks, but it does not guarantee exact, factual recall. Fine-tuned models still hallucinate and cannot cite exact video timestamps.
With RAG, we keep the model frozen and provide the exact lecture transcripts as dynamic context. This guarantees 100% factual accuracy, zero hallucination of course contents, and allows us to provide exact video module numbers and second-by-second timestamps."

Q2. How did you choose your chunk size and overlap?

"Whisper produces short 3- to 6-second sentence fragments. A single sentence lacks technical context for accurate retrieval. Conversely, a 5-minute chunk contains too many mixed topics and exceeds prompt limits.
I grouped Whisper segments into **30- to 60-second chunks (approx. 80–150 words)**. This provides enough context for complete Python code explanations while maintaining precise start/end timestamps."

Q3. How does your Hybrid Retrieval work (BM25 + Dense Vectors)?

"We combine two complementary retrieval approaches:
1. **Dense Semantic Search:** Captures concepts where vocabulary differs (e.g., 'loop over items' matching 'iterate through list').
2. **BM25 Okapi (Lexical):** Excels at exact technical keyword matching (e.g., function names like __init__, f-strings, pip) with Title Weighting.
We normalize both score distributions and blend them using weighted reciprocity: Score = 0.60 * Vector + 0.40 * BM25."

Q4. How do you prevent the LLM from hallucinating?

"We enforce three layers of protection:
1. **Strict Grounding Prompt:** The system prompt explicitly commands the LLM to only answer using the provided lecture context.
2. **Out-of-Syllabus Guardrail:** If the question is outside the syllabus (e.g., 'What is Kubernetes?'), the model states: *'I could not locate documentation for this topic within the indexed course lectures.'*
3. **Low Temperature (0.2):** Keeps inference deterministic, factual, and strictly aligned with the lecture text."

Q5. Why did you choose FastAPI and Groq instead of LangChain or LlamaIndex?

"While frameworks like LangChain are good for rapid prototyping, they introduce heavy abstractions, bloated dependency trees, and high memory overhead.
By writing the RAG pipeline in clean, pure Python with **FastAPI**, the backend boots in under 1 second and consumes less than 80MB RAM. Paired with **Groq Cloud** (LPU architecture delivering 500+ tokens/sec), the entire search and generation lifecycle finishes in **under 1.5 seconds**."

Q6. How did you validate that your RAG system actually works reliably?

"I designed an automated test harness covering **50 diverse questions spanning all 52 modules** (basic syntax, data structures, recursion, file IO, and exercises like KBC).
The test audited context retrieval accuracy, citation correctness, and answer formatting. All 50 queries passed with 100% relevant module citation coverage."

5. Systematic 50-Question Coverage Benchmark

The system was systematically tested against 50 representative questions across the entire curriculum. Every module was verified:

Category	Sample Queries Tested	Modules Mapped	Retrieval Rate
	What is Python?, Variables & Data Types, Typecasting	**Mod 01, 04, 06, 09**	**100% PASS**
Control Flow	**If-Else, Match Case, For Loops, While Loops, Break/Continue**	**Mod 14, 16, 17, 18, 19**	**100% PASS**
	Lists, Tuples, Sets, Dictionaries, String Slicing	**Mod 11, 21, 24, 31, 33**	**100% PASS**
Advanced Core	**Docstrings, Recursion, Lambda Functions, Enumerate**	**Mod 29, 30, 42, 52**	**100% PASS**
	File Open/Read/Write, seek/tell, os Module, Imports	**Mod 44, 46, 49, 50, 51**	**100% PASS**
Error Handling	**Try-Except, Finally keyword, Raising Custom Errors**	**Mod 36, 37, 38**	**100% PASS**
	Calculator, Good Morning Sir, KBC Quiz, Secret Code	**Mod 07, 15, 27, 40**	**100% PASS**
Compound Queries	**What is set and tuple?, Difference between list and tuple**	**Mod 24 & 31, 22 & 24**	**100% PASS**

6. Interview Cheatsheet: What to Emphasize & Avoid

DO EMPHASIZE (Your Strengths)

• Mention that the system is **deployed live on the web** at topic-search-system.onrender.com.
• Highlight the **compound question fix** and the **BM25 title-weighting** you designed.
• Explain how you solved the **512MB RAM constraint** without needing expensive GPU servers.
• Show that you understand **grounding and hallucination prevention**.

DO NOT SAY (Red Flags)

• Don't say *'I just called LangChain or LlamaIndex'* (you built a custom, lightweight pipeline).
• Don't say *'The LLM knows Python from its training'* (emphasize that answers come strictly from the indexed video context).
• Don't claim to have trained a large model from scratch (RAG is an information retrieval system).